Plataforma de Feedback de Aulas

Tech Challenge — Fase 4 · PosTech

Integrante	EBER SPULVEDA LASSO - RM: 371982
Repositório (público)	https://github.com/tech-challenge-fase4/feedback-platform
Página de testes - (no ar até 15/12/2026)	https://d1ujhkrw2v2i80.cloudfront.net
Data de entrega	17/11/2026

> **Para a banca testar agora:** abra a **página de testes** acima. Nela é possível enviar uma avaliação (descrição e nota) e gerar, na hora, o relatório do período escolhido, com a média das notas por dia e o download em PDF. Os e-mails chegam só ao administrador, por isso a página mostra o resultado de cada passo. Detalhes na seção 5.

Sumário

1. Visão geral da solução
2. Modelo de cloud escolhido
3. Arquitetura da solução
4. Documentação das funções
5. Página web de testes
6. Qualidade do código
7. Configuração do ambiente de nuvem
8. Segurança e governança
9. Monitoramento
10. Deploy
11. Custos
12. Vídeos de demonstração
13. Evidências de funcionamento

---

1. Visão geral da solução

Estudantes enviam avaliações das aulas, cada uma com uma descrição e uma nota de 0 a 10. A plataforma:

1. **recebe e grava** cada avaliação pelo endpoint `POST /avaliacao`;
2. **avisa os administradores por e-mail na hora** quando a avaliação é crítica;
3. **envia toda segunda-feira às 08:00** (horário de Brasília) um **relatório semanal** por e-mail, com a média das notas, a quantidade de avaliações por dia e por urgência, e a lista das avaliações;
4. **gera relatórios sob demanda** pelo endpoint `GET /relatorio`, para qualquer período de até 31 dias, com total, média geral, média e quantidade por dia, e quantidade por urgência;
5. oferece uma **página web de testes** (HTTPS) que usa os dois endpoints e exporta o relatório em PDF.

Regra de urgência

O enunciado pede a classificação por urgência mas não define a regra. Adotamos esta:

Nota	Urgência	Ação
0 a 3	**ALTA** (crítica)	E-mail imediato ao administrador + relatórios
4 a 6	**MEDIA**	Só nos relatórios
7 a 10	**BAIXA**	Só nos relatórios

A regra fica em um único lugar do código (`domain/Urgencia.java`). Seguindo o enunciado (*"notificações automáticas ao admin para problemas críticos"*), **só as avaliações críticas geram e-mail imediato**. As demais aparecem no relatório semanal e no relatório sob demanda.

---

2. Modelo de cloud escolhido

Aspecto	Escolha
Modelo de implantação	Nuvem **pública**: AWS, região `us-east-1` (N. Virginia)
Modelo de serviço	**Serverless**: FaaS (**AWS Lambda**) + serviços gerenciados (API Gateway, DynamoDB, SNS, EventBridge Scheduler, S3, CloudFront, CloudWatch, X-Ray)
Estilo de arquitetura	**Orientada a eventos**: uma requisição HTTP, uma gravação no banco e um horário agendado disparam as funções

Por que serverless

- **A carga é irregular:** há picos logo depois das aulas e quase nenhum acesso de madrugada. O Lambda escala automaticamente de zero a muitas execuções simultâneas e volta a zero.
- **Pagamento só pelo uso:** sem requisições, o custo é praticamente zero. Isso importa num projeto acadêmico com créditos limitados (ver seção 11).
- **Sem servidores para operar:** não é preciso gerenciar sistema operacional, patches nem capacidade, e o foco fica na regra de negócio.
- **Responsabilidade única por função:** cada função tem gatilho, permissões, logs e alarmes próprios, e pode ser atualizada e monitorada separadamente.
- **Até a página web é serverless:** arquivos estáticos no S3, entregues pelo CloudFront, sem servidor web.

Responsabilidade compartilhada

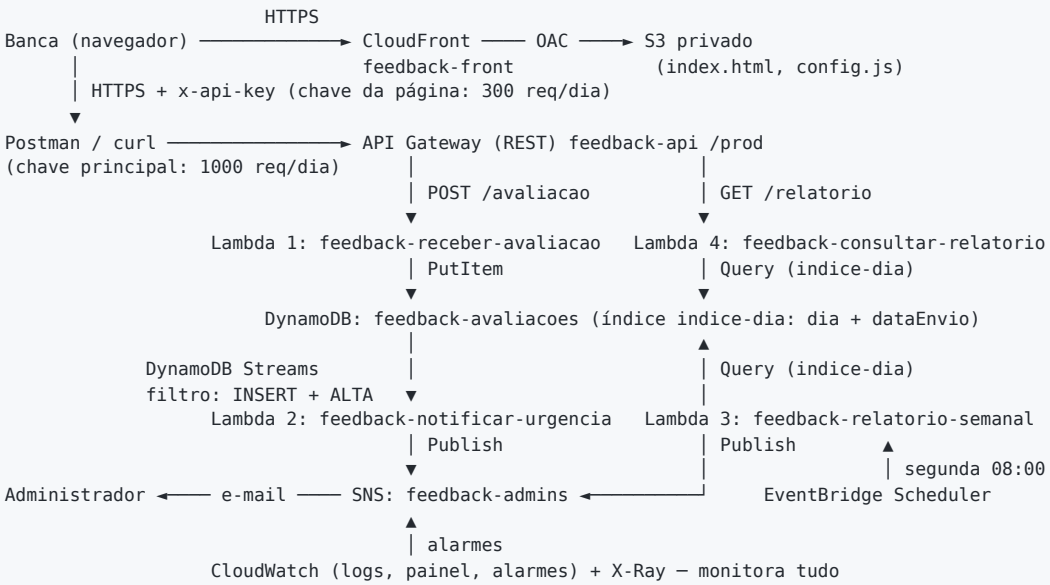
AWS cuida de (segurança da nuvem)	O projeto cuida de (segurança na nuvem)
Hardware, rede física, hipervisor	Permissões IAM (quem acessa o quê)
Runtime Java do Lambda e seus patches	Código das funções e validação de entrada
Disponibilidade e replicação do DynamoDB	Dados gravados e o que vai para os logs
Infraestrutura do API Gateway, SNS, S3 e CloudFront	Chaves de API, limites, CORS, gatilhos e privacidade do bucket

Componentes envolvidos

Componente	Serviço AWS	Papel na solução
Entrada HTTP	**API Gateway** (REST) `feedback-api`, stage `prod`	Expõe `POST /avaliacao` e `GET /relatorio` por HTTPS, com chave de API, plano de uso e CORS
Função 1	**Lambda** `feedback-receber-avaliacao`	Valida a avaliação, calcula a urgência e grava
Banco de dados	**DynamoDB** `feedback-avaliacoes` + índice `indice-dia`	Armazena as avaliações; modo sob demanda; criptografado em repouso
Evento de gravação	**DynamoDB Streams**	Publica cada item novo; um filtro só deixa passar urgência ALTA
Função 2	**Lambda** `feedback-notificar-urgencia`	Envia o alerta de avaliação crítica
Agendamento	**EventBridge Scheduler** `feedback-relatorio-semanal-agenda`	Dispara o relatório toda segunda às 08:00 (America/Sao_Paulo)
Função 3	**Lambda** `feedback-relatorio-semanal`	Consolida a semana e envia o relatório por e-mail
Função 4	**Lambda** `feedback-consultar-relatorio`	Consolida um período sob demanda e devolve JSON (sem e-mail)
Notificações	**SNS** `feedback-admins`	Entrega os e-mails aos administradores
Página web	**S3** `feedback-front-<conta>` (privado) + **CloudFront** `feedback-front`	Hospeda a página de testes com HTTPS; o bucket só é lido pelo CloudFront (OAC)
Observabilidade	**CloudWatch** (logs, painel, alarmes) + **X-Ray**	Monitora as funções e a API e alerta em caso de erro
Governança	**IAM**, **CloudTrail**, **AWS Budgets**	Controle de acesso, auditoria e controle de custos

3. Arquitetura da solução

3.1 Arquitetura de nuvem



3.2 Fluxos

Fluxo 1: registrar avaliação (síncrono)

1. O cliente (página web, Postman ou curl) chama `POST /avaliacao` com o header `x-api-key`.
2. O API Gateway valida a chave e o limite de uso e repassa a requisição à Função 1 (integração *Lambda proxy*).
3. A Função 1 valida os dados, calcula a urgência, grava no DynamoDB e responde **201** com `id`, `urgencia` e `dataEnvio`.

Fluxo 2: alerta de avaliação crítica (assíncrono)

1. A gravação gera um evento no **DynamoDB Streams**.
2. O gatilho da Função 2 tem um **filtro** (`eventName = INSERT` e `urgencia = ALTA`). Avaliações não críticas nem chegam a executar a função, o que economiza execuções.
3. A Função 2 publica no **SNS**, que envia o e-mail **[URGENTE]** com descrição, urgência e data de envio.

Fluxo 3: relatório semanal por e-mail (agendado)

1. O **EventBridge Scheduler** executa a Função 3 toda segunda às 08:00 (`cron(0 8 ? * MON *)`, fuso `America/Sao_Paulo`).
2. A Função 3 consulta os **7 dias anteriores** (segunda a domingo) no índice `indice-dia`, dia a dia, sem varrer a tabela inteira.
3. Ela calcula total, média, quantidade por dia e por urgência e envia o relatório pelo **SNS**.

Fluxo 4: relatório sob demanda (síncrono)

1. O cliente chama `GET /relatorio?inicio=aaaa-mm-dd&fim=aaaa-mm-dd` com o header `x-api-key`. Sem parâmetros, o período é o dos últimos 7 dias, **incluindo hoje**, para que uma avaliação recém-enviada já apareça.
2. O API Gateway valida a chave e o limite e repassa à Função 4.
3. A Função 4 valida o período (no máximo 31 dias), consulta o índice `indice-dia` dia a dia e responde **200** com os números agregados.
4. Na página web, o resultado vira cartões, uma tabela com a média por dia e, se o usuário quiser, um PDF.

As Funções 3 e 4 usam **o mesmo caso de uso de consolidação** (`ConsultarRelatorioUseCase`). Por isso o e-mail semanal e o relatório sob demanda dão sempre os mesmos números para o mesmo período.

3.3 Modelo de dados (DynamoDB)

Tabela `feedback-avaliacoes`, chave de partição `id` (String):

Atributo	Tipo	Exemplo	Observação
`id`	String	`3f2c…` (UUID)	Chave primária
`descricao`	String	`Aula excelente, muito didática`	1 a 1000 caracteres
`nota`	Number	`9`	Inteiro de 0 a 10
`urgencia`	String	`BAIXA`	Calculada pela nota
`dataEnvio`	String	`2026-11-02T13:10:00Z`	Instante UTC (ISO-8601)
`dia`	String	`2026-11-02`	Data local de São Paulo; partição do índice

Índice global `indice-dia`: partição `dia` + ordenação `dataEnvio`. Permite buscar as avaliações de um dia com `Query`, que é mais barata e rápida que `Scan`. Os dois relatórios usam só esse índice.

3.4 Arquitetura de software: Clean Architecture + SOLID

O código é um único projeto Java 25 / Quarkus, organizado em camadas:

```
src/main/java/br/com/techchallenge/feedback/
├── domain/              Regras de negócio puras (só Java)
│                        Avaliacao, Urgencia, PeriodoRelatorio (até 31 dias), RelatorioSemanal,
│                        AvaliacaoInvalidaException, PeriodoInvalidoException
├── application/
│   ├── port/            Interfaces (portas): AvaliacaoRepository, NotificacaoGateway, GeradorDeId
│   ├── usecase/         Um caso de uso por operação:
│   │                    RegistrarAvaliacao, NotificarAvaliacaoCritica,
│   │                    ConsultarRelatorio, GerarRelatorioSemanal (= ConsultarRelatorio + envio)
│   └── formatacao/      Texto dos e-mails: MensagemUrgenciaFormatter, RelatorioFormatter
├── infrastructure/      Implementações técnicas das portas
│                        DynamoDbAvaliacaoRepository, SnsNotificacaoGateway, UuidGeradorDeId
│                        e ConfiguracaoCasosDeUso (montagem / composition root)
└── entrypoint/lambda/   Os 4 handlers Lambda (só traduzem evento ↔ caso de uso)
                         receber/, notificar/, relatorio/, consultar/ e http/ (resposta + CORS)

frontend/                Página web estática: index.html e config.example.js
```

Regra de dependência: `entrypoint` e `infrastructure` → `application` → `domain`. O domínio e os casos de uso **não importam** AWS SDK, Quarkus nem Jackson. Assim, as regras de negócio podem ser testadas e mantidas sem depender da nuvem.

Princípio SOLID	Onde aparece
S – Responsabilidade única	Cada função Lambda tem um único papel: a Função 3 envia o relatório, a Função 4 só o consulta. Cada caso de uso faz uma operação. Os handlers só traduzem eventos. O cálculo (`RelatorioSemanal`) é separado da formatação (`RelatorioFormatter`). O formato do DynamoDB fica só em `AvaliacaoItemMapper`
O – Aberto/fechado	O relatório sob demanda foi acrescentado **sem alterar** o domínio de avaliação nem as portas: um caso de uso novo, um handler novo e um método de cálculo novo. Um novo canal de aviso (ex.: Slack) seria uma nova implementação de `NotificacaoGateway`
L – Substituição de Liskov	Qualquer implementação das portas (DynamoDB, SNS ou em memória) pode ser usada pelos casos de uso
I – Segregação de interfaces	Portas pequenas, com 1 ou 2 métodos cada
D – Inversão de dependência	Os casos de uso dependem de interfaces; `ConfiguracaoCasosDeUso` entrega as implementações concretas

Reuso por composição: `GerarRelatorioSemanalUseCase` não repete a consolidação; ele usa o `ConsultarRelatorioUseCase` e só acrescenta o envio do e-mail.

---

4. Documentação das funções

As 4 funções usam o **mesmo pacote** (`target/function.zip` , gerado pelo Quarkus) e o **mesmo handler de entrada**:

```
io.quarkus.amazon.lambda.runtime.QuarkusStreamHandler::handleRequest
```

A variável de ambiente `QUARKUS_LAMBDA_HANDLER` escolhe qual classe `@Named` cada função executa. Na AWS são **4 recursos independentes**, cada um com gatilho, role IAM, logs e alarme próprios.

Configuração comum: runtime **Java 25**, arquitetura **arm64**, **1024 MB** de memória, timeout de **30 s**, rastreamento **X-Ray** ativo, armazenamento efêmero de 512 MB (padrão).

4.1 `feedback-receber-avaliacao`: registrar avaliação

Responsabilidade	Validar a entrada, calcular a urgência e gravar a avaliação
Gatilho	API Gateway: `POST /avaliacao` (header `x-api-key` obrigatório)
`QUARKUS_LAMBDA_HANDLER`	`receberAvaliacao`
Classes	`ReceberAvaliacaoHandler` → `RegistrarAvaliacaoUseCase` → `DynamoDbAvaliacaoRepository`
Variáveis de ambiente	`TABELA_AVALIACOES=feedback-avaliacoes`, `CORS_ORIGEM=<URL do CloudFront>`
Permissão IAM	Somente `dynamodb:PutItem` na tabela `feedback-avaliacoes`

Requisição

```
POST /prod/avaliacao
Content-Type: application/json
x-api-key: <chave>

{ "descricao": "Aula excelente, muito didática", "nota": 9 }
```

Respostas

Status	Quando	Corpo
201	Avaliação gravada	`{"id": "...", "urgencia": "BAIXA", "dataEnvio": "2026-11-02T13:10:00Z", "mensagem": "Avaliação recebida com sucesso"}`
400	Corpo ausente ou JSON inválido; `descricao` ausente, vazia ou com mais de 1000 caracteres; `nota` ausente, fora de 0–10, decimal ou em texto (`"9"`)	`{"erro": "<motivo>"}`
403	Sem `x-api-key` ou chave inválida (barrado pelo API Gateway)	`{"message": "Forbidden"}`
429	Limite de taxa ou cota do plano de uso excedido (barrado pelo API Gateway)	`{"message": "..."}`
500	Erro inesperado	`{"erro": "Erro interno ao processar a avaliação"}`

4.2 `feedback-notificar-urgencia`: alerta de avaliação crítica

Responsabilidade	Avisar os administradores por e-mail sobre avaliações críticas
Gatilho	DynamoDB Streams da tabela (*New image*), lote de 10, posição *Latest*, 3 tentativas, filtro `{"eventName":["INSERT"],"dynamodb":{"NewImage":{"urgencia":{"S":["ALTA"]}}}}`
`QUARKUS_LAMBDA_HANDLER`	`notificarUrgencia`
Classes	`NotificarUrgenciaHandler` → `NotificarAvaliacaoCriticaUseCase` → `SnsNotificacaoGateway`
Variáveis de ambiente	`TOPICO_ARN=<ARN do tópico feedback-admins>`
Permissões IAM	`sns:Publish` no tópico; `DescribeStream`, `GetRecords` e `GetShardIterator` no stream da tabela; `ListStreams`
Saída	E-mail **“[URGENTE] Nova avaliacao critica recebida”** com descrição, urgência e data de envio

A regra “só avisa se a urgência for ALTA” está **no caso de uso**, além do filtro do gatilho. Assim ela é testável e não depende só da configuração da AWS.

4.3 `feedback-relatorio-semanal`: relatório semanal por e-mail

Responsabilidade	Consolidar as avaliações da semana e enviar o relatório
Gatilho	EventBridge Scheduler `feedback-relatorio-semanal-agenda`: `cron(0 8 ? * MON *)`, fuso `America/Sao_Paulo`, payload `{}`
`QUARKUS_LAMBDA_HANDLER`	`relatorioSemanal`
Classes	`RelatorioSemanalHandler` → `GerarRelatorioSemanalUseCase` → `ConsultarRelatorioUseCase` + `SnsNotificacaoGateway`
Variáveis de ambiente	`TABELA_AVALIACOES=feedback-avaliacoes`, `TOPICO_ARN=<ARN do tópico>`
Permissões IAM	`dynamodb:Query` **somente** no índice `indice-dia`; `sns:Publish` no tópico
Período	Os 7 dias anteriores à execução (na segunda: de segunda a domingo da semana anterior)
Saída	E-mail **"Relatorio semanal de feedbacks - dd/MM/aaaa a dd/MM/aaaa"** com total, média das notas, quantidade por dia, quantidade por urgência e a lista das avaliações (descrição, urgência e data de envio)

Para execução manual fora do agendamento, a função aceita `{"incluirHoje": true}`, que cobre os 6 dias anteriores mais o dia atual.

4.4 `feedback-consultar-relatorio`: relatório sob demanda

Responsabilidade	Consolidar as avaliações de um período e devolver os números, **sem enviar e-mail**
Gatilho	API Gateway: `GET /relatorio` (header `x-api-key` obrigatório)
`QUARKUS_LAMBDA_HANDLER`	`consultarRelatorio`
Classes	`ConsultarRelatorioHandler` → `ConsultarRelatorioUseCase` → `DynamoDbAvaliacaoRepository`
Variáveis de ambiente	`TABELA_AVALIACOES=feedback-avaliacoes`, `CORS_ORIGEM=<URL do CloudFront>`
Permissão IAM	Somente `dynamodb:Query` no índice `indice-dia` (sem SNS, sem escrita)
Período	`inicio` e `fim` (`aaaa-mm-dd`), no máximo **31 dias**; sem parâmetros, os últimos 7 dias incluindo hoje

Requisição

```
GET /prod/relatorio?inicio=2026-11-02&fim=2026-11-08
x-api-key: <chave>
```

Resposta 200: só números agregados; o texto das avaliações **não** é devolvido:

```
{
  "inicio": "2026-11-02",
  "fim": "2026-11-08",
  "total": 21,
  "media": 6.57,
  "porUrgencia": { "ALTA": 3, "MEDIA": 5, "BAIXA": 13 },
  "porDia": [
    { "dia": "2026-11-02", "quantidade": 3, "media": 6.33 },
    { "dia": "2026-11-03", "quantidade": 0, "media": null }
  ]
}
```

Em `porDia`, `media: null` indica **dia sem avaliações**, que é diferente de média 0 (nota 0 é uma nota válida).

Respostas de erro

Status	Quando	Corpo
400	Só um dos parâmetros; data fora do formato `aaaa-mm-dd`; início depois do fim; período maior que 31 dias	`{"erro": "<motivo>"}`
403	Sem `x-api-key` ou chave inválida	`{"message": "Forbidden"}`
429	Limite de taxa ou cota excedido	`{"message": "..."}`
500	Erro inesperado	`{"erro": "Erro interno ao gerar o relatório"}`

4.5 Variáveis de ambiente (resumo)

Variável	F1	F2	F3	F4	Valor
`QUARKUS_LAMBDA_HANDLER`	✔	✔	✔	✔	`receberAvaliacao` / `notificarUrgencia` / `relatorioSemanal` / `consultarRelatorio`
`TABELA_AVALIACOES`	✔		✔	✔	`feedback-avaliacoes`
`TOPICO_ARN`		✔	✔		ARN do tópico `feedback-admins`
`CORS_ORIGEM`	✔			✔	URL da página (`https://<id>.cloudfront.net`), sem barra no final
`JAVA_TOOL_OPTIONS`	✔	✔	✔	✔	`-XX:+TieredCompilation -XX:TieredStopAtLevel=1` (acelera a partida a frio)

---

5. Página web de testes

Endereço: https://d1ujhkrw2v2i80.cloudfront.net/ , disponível até **15/12/2026**.

Navegador recomendado: Google Chrome atualizado, no computador ou no celular. A página usa só recursos padrão da web, então deve funcionar também nas versões atuais do Edge, Firefox e Safari.

A página existe para que a banca possa **testar a solução em tempo real**, sem Postman e sem acesso ao e-mail do administrador.

Parte	O que faz
1. Enviar uma avaliação	Campos **descrição** (até 1000 caracteres, com contador) e **nota** (0 a 10). Chama `POST /avaliacao` e mostra a urgência calculada pelo servidor. Se for ALTA, informa que o administrador foi avisado por e-mail
2. Gerar relatório	Período com padrão nos últimos 7 dias, incluindo hoje (máximo de 31 dias). Chama `GET /relatorio` e mostra total, média geral, quantidade por urgência e uma tabela **por data** com quantidade, **média das notas** e uma barra colorida pela faixa de urgência
Baixar PDF	Gera o PDF do relatório pelo próprio navegador (“Salvar como PDF”). Os formulários ficam ocultos e o nome sugerido é `relatorio-feedbacks-<inicio>-a-<fim>.pdf`
Como funciona	Resumo do fluxo da solução (página → API Gateway → Lambda → DynamoDB → SNS / Scheduler) e da regra de urgência

Roteiro sugerido para a banca:

1. Envie uma avaliação com nota alta (ex.: 9) e outra crítica (ex.: 1).
2. Clique em **Gerar relatório**: as duas aparecem na linha de hoje e nos totais.
3. Clique em **Baixar PDF**.
4. Opcional: escolha um período maior (ex.: de 30/10 até hoje) para ver as avaliações do período de execução (seção 13).

Decisões técnicas

- **Hospedagem serverless:** arquivos estáticos (`index.html` e `config.js`) num bucket **S3 privado**, entregues pelo **CloudFront** com **HTTPS**. O acesso direto ao S3 é negado; só o CloudFront lê o bucket, por *Origin Access Control* (OAC).
- **Sem dependências externas:** HTML, CSS e JavaScript puros, sem bibliotecas nem CDNs de terceiros.
- **PDF no navegador:** gerar PDF na Lambda exigiria uma biblioteca a mais, aumentaria a partida a frio e o custo. O navegador já faz isso de graça.
- **Chave própria e limitada:** numa página estática a chave de API é visível a quem abrir o código-fonte. Por isso a página usa uma chave **separada** (`feedback-chave-front`), com plano de uso próprio: **2 req/s, rajada de 5 e 300 req/dia**. Ela pode ser revogada sem afetar o resto (seção 8.3).
- **Privacidade:** o relatório sob demanda devolve só números agregados. O texto das avaliações não é exposto pela página.
- **Responsiva e segura:** funciona no celular; todo conteúdo vindo da API é inserido como texto (`textContent`), sem risco de injeção de HTML.
- **Partida a frio explicada:** a primeira chamada depois de um tempo parado pode levar alguns segundos, porque o Lambda inicia a JVM. Se passar de 3 s, a página avisa, para ninguém achar que travou.
- **Validação também no navegador:** o limite de 31 dias e a ordem das datas são conferidos antes da chamada, para não gastar a cota com requisições que seriam recusadas.

---

6. Qualidade do código

- **Clean Architecture + SOLID** (seção 3.4): regras de negócio isoladas da infraestrutura.
- **Entidade sempre válida:** o construtor de `Avaliacao` valida tudo (nota de 0 a 10, descrição de 1 a 1000 caracteres, urgência coerente com a nota). Não existe avaliação inválida em memória. Da mesma forma, `PeriodoRelatorio` não aceita início depois do fim nem mais de 31 dias.
- **Validação estrita da entrada:** `"9"` (texto) e `9.5` (decimal) são recusados como nota, em vez de convertidos silenciosamente. Datas fora do formato `aaaa-mm-dd` são recusadas com 400.
- **Erros tratados com status HTTP corretos** (400, 500) e mensagens claras em português.
- **Privacidade nos logs e nas respostas:** o texto da avaliação **não** vai para os logs (registram-se apenas `id` , nota e urgência) e **não** é devolvido pelo `GET /relatorio` .
- **Sem duplicação:** o relatório semanal e o sob demanda usam o mesmo caso de uso de consolidação.
- **Documentação no código:** todas as classes têm Javadoc explicando a responsabilidade e o papel na arquitetura.
- **Nomes do domínio em português**, alinhados ao enunciado (`Avaliacao` , `Urgencia` , `RelatorioSemanal`).

7. Configuração do ambiente de nuvem

Todos os recursos estão em `us-east-1` e foram criados pelo Console AWS, nesta ordem.

#	Recurso	Configuração
1	**DynamoDB** `feedback-avaliacoes`	Partição `id` (String); capacidade **On-demand**; criptografia em repouso (*owned by Amazon DynamoDB*); índice global `indice-dia` (`dia` + `dataEnvio`, projeção *All*); **Streams** ligado com *New image*
2	**SNS** `feedback-admins`	Tópico *Standard*; assinatura de e-mail do administrador confirmada
3	**IAM** (4 roles)	`feedback-role-receber`, `feedback-role-notificar`, `feedback-role-relatorio`, `feedback-role-consultar`: cada uma com `AWSLambdaBasicExecutionRole` + `AWSXRayDaemonWriteAccess` + 1 política *inline* de menor privilégio (seção 8.2)
4	**Lambda** (4 funções)	Java 25, arm64, 1024 MB, 30 s, X-Ray ativo, handler `QuarkusStreamHandler`, variáveis de ambiente da seção 4.5
5	**Gatilho do stream**	DynamoDB → `feedback-notificar-urgencia`: lote 10, *Latest*, 3 tentativas, filtro `INSERT` + `ALTA`
6	**API Gateway** `feedback-api`	REST regional; `POST /avaliacao` → Função 1 e `GET /relatorio` → Função 4, ambos com *Lambda proxy integration* e **API key required**; CORS habilitado nos dois recursos (origem = URL do CloudFront; *Gateway responses* 4XX/5XX com CORS; `OPTIONS` sem chave); stage `prod`
7	**Plano de uso** `feedback-plano-de-uso`	10 req/s, rajada de 20, cota de 1000 req/dia; chave `feedback-chave-demo` (Postman e vídeo)
8	**Plano de uso** `feedback-plano-front`	2 req/s, rajada de 5, cota de 300 req/dia; chave `feedback-chave-front` (página web)
9	**EventBridge Scheduler** `feedback-relatorio-semanal-agenda`	Recorrente, `cron(0 8 ? * MON *)`, `America/Sao_Paulo`, *flexible time window* desligado, alvo `feedback-relatorio-semanal`, payload `{}`
10	**S3** `feedback-front-<conta>`	*Block all public access* ligado; ACLs desabilitadas; SSE-S3; política que permite `s3:GetObject` **somente** à distribuição CloudFront
11	**CloudFront** `feedback-front`	Origem S3 com **OAC**; *Redirect HTTP to HTTPS*; cache *CachingOptimized*; objeto raiz `index.html`
12	**CloudWatch**	Retenção de logs de 30 dias, painel `feedback-painel` e 5 alarmes (seção 9)

Por que REST API e não HTTP API no API Gateway: só a REST API oferece **chave de API e plano de uso** (limite de taxa e cota), que fazem parte da proteção do endpoint público.

Por que arm64: no Lambda, o processador ARM (Graviton) custa menos por milissegundo e tem bom desempenho para Java, sem nenhuma mudança no código.

Por que S3 + CloudFront e não o "Static website hosting" do S3: o endpoint de site do S3 só tem HTTP e exige bucket público. Com o CloudFront, a página tem HTTPS e o bucket continua privado.

---

8. Segurança e governança

8.1 Acesso à conta

- **MFA** no usuário root e nos usuários IAM.
- O **root não é usado no dia a dia**. O acesso é feito pelo usuário IAM `admin-techchallenge`, com login pela URL do alias da conta.
- **Grupos IAM:** `Administradores` (`AdministratorAccess`) para quem altera recursos e `Observadores` (`ReadOnlyAccess`) para quem só consulta.
- **CloudTrail** registra todas as ações na conta (histórico de 90 dias), garantindo rastreabilidade.
- **AWS Budgets** com orçamento "Zero spend": alerta por e-mail ao primeiro gasto.

8.2 Menor privilégio por função

Cada função tem **uma role própria**, com somente as ações e os recursos de que precisa. Tudo o que não está explicitamente permitido é negado.

Role	Permite	Não permite
`feedback-role-receber`	`dynamodb:PutItem` na tabela	Ler, apagar, consultar, publicar no SNS
`feedback-role-notificar`	`sns:Publish` no tópico; leitura do stream da tabela	Gravar, alterar ou consultar a tabela
`feedback-role-relatorio`	`dynamodb:Query` **só no índice** `indice-dia`; `sns:Publish` no tópico	Gravar, apagar, `Scan`, `GetItem`, ler o stream
`feedback-role-consultar`	`dynamodb:Query` **só no índice** `indice-dia`	Gravar, apagar, `Scan`, `GetItem`, ler o stream, **publicar no SNS**

Exemplo, a política da Função 4:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsultarIndicePorDia",
      "Effect": "Allow",
      "Action": "dynamodb:Query",
      "Resource": "arn:aws:dynamodb:us-east-1:<ACCOUNT_ID>:table/feedback-avaliacoes/index/indice-dia"
    }
  ]
}
```

A Função 4 é a única exposta para leitura pela internet. Mesmo que alguém abusasse do endpoint, ele só conseguiria ler números agregados: não poderia gravar, apagar nem disparar e-mails.

8.3 Proteção da API

- **HTTPS** obrigatório (padrão do API Gateway).
- **Chave de API** (`x-api-key`) em todos os métodos: sem ela, a requisição é barrada com **403** antes de chegar à função.
- **Duas chaves, dois planos de uso:**

Chave	Uso	Limite	Exposição
`feedback-chave-demo`	Postman, curl, vídeo	10 req/s, rajada 20, 1000 req/dia	Secreta
`feedback-chave-front`	Página web	2 req/s, rajada 5, 300 req/dia	Visível no código da página (inevitável numa página estática)

A chave da página pode ser **desativada sozinha**, sem afetar o Postman, os e-mails nem os relatórios.

- **CORS restrito:** só a origem da página (`https://<id>.cloudfront.net`) pode ler as respostas no navegador. Não é usado `*`. Postman e curl não dependem de CORS.
- **Validação estrita** da entrada nas funções (seções 4.1 e 4.4).

8.4 Proteção dos dados

- DynamoDB **criptografado em repouso** (padrão, sem custo).
- Avaliações **anônimas**: nenhum dado pessoal do estudante é coletado.
- O texto das avaliações **não é registrado nos logs** e **não é devolvido pelo relatório sob demanda**. Só o administrador o recebe, no e-mail semanal.
- Bucket da página **privado**, acessível só pelo CloudFront.
- As chaves de API e o número da conta **não estão no repositório**: o `frontend/config.js` (com a chave da página) está no `.gitignore`, e o repositório traz apenas o modelo `config.example.js`.

---

9. Monitoramento

O quê	Configuração
Logs	CloudWatch Logs, um grupo por função (`/aws/lambda/feedback-*`), retenção de **30 dias**
Consulta de erros	CloudWatch Logs Insights sobre os 4 grupos: `filter @message like /ERROR|Erro|Exception/`
Painel	`feedback-painel`: invocações, erros e duração média das 4 funções; requisições, 4XX e 5XX da API
Alarmes	`feedback-erros-receber`, `feedback-erros-notificar`, `feedback-erros-relatorio`, `feedback-erros-consultar` (métrica `Errors` ≥ 1 em 5 min) e `feedback-api-5xx` (`5XXError` ≥ 1 em 5 min). Todos notificam o administrador por e-mail via SNS `feedback-admins`
Rastreamento	**AWS X-Ray** ativo nas funções e na API: mostra o caminho API Gateway → Lambda → DynamoDB e a latência de cada etapa, inclusive das chamadas feitas pela página
Uso da API	API Gateway → *Usage plans*: consumo diário de cada chave (útil para acompanhar o uso da página pela banca)
Auditoria	CloudTrail (quem alterou o quê na conta)
Custos	AWS Budgets com alerta por e-mail

Os administradores recebem assim **dois tipos de notificação automática**:

- **de negócio:** avaliação crítica (Função 2);
- **técnica:** erro em uma função ou na API (alarmes do CloudWatch).

---

10. Deploy

10.1 Deploy automatizado das funções (GitHub Actions)

A cada `git push` na branch `main`, o workflow `.github/workflows/deploy.yml`:

1. compila o projeto com Java 25 e gera o `target/function.zip`;
2. autentica na AWS por **OIDC**, sem chaves de acesso guardadas no GitHub;
3. atualiza o código das **4 funções** (`aws lambda update-function-code`) e espera cada uma ficar pronta.

Se a compilação falhar, os passos de deploy não executam: código quebrado não chega à AWS.

A role usada pelo GitHub (`github-actions-deploy-feedback`) só pode ser assumida por **este repositório e esta branch**, e só tem permissão de atualizar o código das funções `feedback-*`.

10.2 Publicação da página

A página muda raramente e depende do `config.js`, que não fica no repositório. Por isso a publicação é manual: upload de `index.html` e `config.js` no bucket e invalidação `/*` no CloudFront.

10.3 Testar a API após o deploy

```
# Registrar uma avaliação
curl -i -X POST "https://<API_ID>.execute-api.us-east-1.amazonaws.com/prod/avaliacao" \
  -H "Content-Type: application/json" \
  -H "x-api-key: <CHAVE>" \
  -d '{"descricao":"Aula excelente, muito didática","nota":9}'

# Relatório sob demanda de um período
curl -i "https://<API_ID>.execute-api.us-east-1.amazonaws.com/prod/relatorio?inicio=2026-10-30&fim=2026-11-15" \
  -H "x-api-key: <CHAVE>"
```

A pasta `postman/` do repositório traz uma **collection do Postman** com:

- avaliações variadas (todas as faixas de nota), organizadas por dia do plano de execução;
- os cenários de erro (400 e 403);
- a pasta **Relatório sob demanda** (período padrão, período do plano, datas inválidas, mais de 31 dias, sem chave).

Todas as requisições têm testes automáticos de status e conteúdo.

---

11. Custos

Como os créditos de nuvem são limitados, todo o projeto foi pensado para ficar dentro dos níveis gratuitos e ter um **teto de gasto**, mesmo em caso de uso indevido.

O que limita o custo:

- **Cota diária nos dois planos de uso:** no máximo 1.300 requisições por dia, somando as duas chaves. Acima disso o API Gateway responde 429 sem executar nenhuma função.
- **Filtro no gatilho do stream:** só avaliações críticas executam a Função 2.
- `Query` **por dia no índice**, nunca `Scan`, e no máximo **31 dias** por relatório.
- **PDF gerado no navegador**, sem processamento na nuvem.
- **arm64** no Lambda (mais barato por milissegundo).
- **Retenção de logs de 30 dias.**
- **AWS Budgets** alertando ao primeiro gasto.

Estimativa no pior caso (as duas cotas esgotadas todos os dias, de 29/10 a 15/12/2026, cerca de 61 mil requisições):

Serviço	Pior caso	Uso real esperado
API Gateway REST	~US$ 0,21	centavos
Lambda (4 funções)	dentro do nível gratuito	US$ 0
DynamoDB sob demanda	~US$ 0,10	centavos
SNS e-mail (se todas as avaliações fossem críticas)	~US$ 1,20	US$ 0 (1.000 e-mails grátis por mês)
S3 + CloudFront	dentro do nível gratuito	US$ 0
Total	**menos de US$ 2**	**centavos**

Depois de 15/12/2026, todos os recursos serão removidos.

---

12. Vídeos de demonstração

#	Conteúdo	Link
1	**Apresentação geral e em funcionamento:** modelo de cloud, arquitetura, configurações no console AWS (funções, roles IAM, API Gateway, DynamoDB, SNS, Scheduler, S3 e CloudFront), segurança e monitoramento. envio de avaliações pela API, gravação no DynamoDB, e-mail de avaliação crítica, cenários de erro (400/403), logs e X-Ray	https://youtu.be/QviVpkKXXnk

---

13. Evidências de funcionamento

Entre 30/10 e 15/11/2026 a aplicação recebeu avaliações reais pelo endpoint público, uma rodada por dia, com notas de todas as faixas. Os relatórios semanais foram gerados pelo próprio agendamento, sem disparo manual.

Essas mesmas avaliações podem ser conferidas pela banca na **página de testes**, gerando o relatório de **30/10/2026 a 15/11/2026**. Os números batem com os dos e-mails semanais, porque os dois relatórios usam a mesma consolidação.